PROJECT HOOVER · TODDLER WRITER · ANALYSIS REPORT

The Local Model Can't Share the Race-Night PC

Llama 3.2 against Claude Haiku on the Oklahoma spectator PC. The short brief makes the local model 2.5 times faster than the cloud, but F1 25 and OBS leave no room for it on the 8 GB card.

Hoover_Local_LLM_Bench_Findings_V1_26OCT26 · Tested 26 OCT 26 on the Oklahoma PC (i5-9400F, RTX 3070 8 GB, 48 GB RAM) · Instrument: T13 LLM Bench V1 · Data: 21 OCT race-night state blobs (HOOVER_20261021_223105_s01)

BOTTOM LINE

On race night, Toddler writes with cloud Haiku and falls back to templates. The local model stays in the design, switched off on this PC. With the game off, llama3.2 answered a Toddler-style brief in 0.35 s, faster than any cloud call. With F1 25 and OBS running, the same request took 10 s, because the card was already full and the model was pushed onto the CPU. Cloud ran at 0.89 s either way.

0.35 s
Local, short brief, game off. Median per line, 40 lines.

10.1 s
Local, same brief, F1 25 + OBS running. Median per line, 20 lines.

0.89 s
Cloud Haiku, game on. It didn't notice the game was running.

1. The question

The 26 OCT synthesis paper (*Local Ollama vs Cloud Haiku: Complete Synthesis, Peer Review & Controlled A/B Test Plan*) showed that Baby Hoover was unfair to the local model. It got a 0.6 s cutoff where cloud got 2.2 s, two requests were sent at once to one GPU, the context size was never set, and the prompts were sized for a cloud model. What the paper could not show was whether the local model would work for Toddler. Baby's numbers were measured with the wrong prompts and, most likely, with the game closed.

So the test asked one plain question: **can a local model write good-enough lines, fast enough, on the Oklahoma PC while F1 25 and OBS are running?**

2. How it was tested

We built a standalone instrument, `tools/T13_LLM_Bench_V1_26OCT26.py`, instead of patching the frozen Baby code. It imports nothing from Hoover, so it can't go out of date as Toddler changes. It takes real story beats from the 21 OCT race, builds each request two ways, and sends it to both models.

Format	What the model is given	Size (median)
Baby	Baby Hoover's own single-line request: the full JSON state blob under Baby's system prompt.	≈11,000 chars · ≈3,100 tokens
Toddler	A short brief worked out in Python first: who is speaking, the job (call it, why, what it means), three or four facts, the names allowed, the last three lines.	≈1,350 chars · ≈360 tokens

The bench fixes each of the faults the synthesis found:

- one request at a time;
- no cutoff, so every call runs to completion and is timed;
- the context size set to match each format (8,192 tokens for Baby, 2,048 for Toddler);
- Ollama's own timings logged (reading the prompt, writing the line, loading the model);
- local and cloud take turns going first on each beat.

GPU memory and load were sampled every second with `nvidia-smi`.

Run	Conditions	Scope
BENCH_20261026_230257	Game closed, OBS closed	40 beats × both formats × both models (160 calls)
BENCH_20261026_231639	F1 25 race on screen (spectating), 60 fps cap, OBS open	Baby format only. Stopped after 10 beats when lines took more than 20 s.
BENCH_20261026_232316	F1 25 race on screen (spectating), 60 fps cap, OBS open, recording from about call 34	Toddler format, 20 beats × both models (40 calls)

3. Results

Format	Model	Game	n	Median	p90	Read prompt	Write line	Rule breaks
Baby	Local llama3.2	off	40	1.09 s	1.20 s	0.52 s	0.27 s	10%
Baby	Cloud Haiku	off	40	0.90 s	1.04 s	–	–	18%
Toddler	Local llama3.2	off	40	**0.35 s**	**0.57 s**	0.05 s	0.25 s	28%
Toddler	Cloud Haiku	off	40	0.86 s	0.99 s	–	–	8%
Baby	Local llama3.2	on	10	**23.4 s**	**28.7 s**	6.0 s	17.2 s	20%
Baby	Cloud Haiku	on	10	0.92 s	1.00 s	–	–	10%
Toddler	Local llama3.2	on	20	**10.1 s**	**12.3 s**	1.16 s	8.72 s	25%
Toddler	Cloud Haiku	on	20	0.89 s	0.96 s	–	–	15%

Wall seconds per call, one call at a time. "Read prompt" and "Write line" are Ollama's own timings; the cloud doesn't report them. Model reload: 6 s with the game off, 39 to 40 s with the game on.

Finding 1: the short brief is what makes a local model viable

Cutting the prompt from about 3,100 tokens to 360 made the local model three times faster: 1.09 s down to 0.35 s. Reading the prompt went from 0.52 s to 0.05 s. The cloud barely moved (0.90 s to 0.86 s),

because a data centre reads long prompts quickly. Baby's large state blob was the main reason local looked slow, and the synthesis paper's forecast of 0.15 to 0.35 s for a Toddler-sized prompt was correct with the game closed.

Finding 2: with the game running, there is no room on the card

GPU, while the bench sat idle	Memory in use	GPU load
Game and OBS closed	3.3 GB of 8 GB	14%
F1 25 race on screen and OBS open (Baby-format run)	7.8 GB of 8 GB	77%
F1 25 race on screen and OBS open (Toddler-format run)	7.5 GB of 8 GB	42%

With the card nearly full, Ollama couldn't hold llama3.2 entirely on the GPU, and the timings show it:

- Writing a 27-token line went from 0.25 s to 8.7 s, which is CPU speed.
- Reloading the model went from 6 s to 39 s.

The smaller context in the Toddler format helped (10 s instead of 23 s) but nowhere near enough. The synthesis paper's VRAM budget assumed F1 25 and OBS would stay under about 5.1 GB. On this PC they didn't.

Finding 3: the stream pays too

Dustin watched both game-on runs:

- The game hitched briefly at the start of the Baby-format run. That lines up with the 40 s model load fighting the game for memory.
- During the Toddler-format run the game held steady, but **OBS stuttered**.

On race night OBS is the stream, so a smooth game over a choppy encode is still a failure. Even a slow local model running mostly on the CPU took resources the broadcast needed.

Finding 4: the cloud doesn't care what the PC is doing

Haiku ran at 0.86 to 0.92 s in every condition. Its p90 never passed 1.04 s, and no call failed across 110 cloud calls. On this machine, it's the only writer whose speed doesn't depend on what else is running.

4. Line quality

The rule-break counts look worse for local than they really are. Most of local's breaks were mechanical:

- putting *"Lead:"* in front of the line (9 times across runs);
- running past the word limit.

Code can strip the label and cut to the first sentence, and the cloud made the same kinds of mistake (seven speaker labels in the Baby format, plus dashes).

The real gap is accuracy. With the game off, on the Toddler brief, the local model regularly stated things the facts did not support:

Local line (Toddler brief)	The problem
Dustin is moving up the inside of Mike as they enter Turn One…	Invents the move and the corner; the facts gave a gap, not a pass attempt.
Alex is now side by side with Alonso, just a tenth of a second behind…	Invents the side-by-side and shrinks the gap.
…if it holds, Alex would be on the podium…	Wrong position consequence.
Norris is currently in fourth place, thirty-three point seven seconds behind leader Dustin…	Wrong leader, wrong gap.
…as he makes a bold move on Cade Norris…	Merges two drivers into one name.

The cloud also strayed, but into opinion rather than facts: *"championship hopes"*, *"redemption arc"*. A booth can live with a bit of extra colour. A wrong position or a merged name is the kind of error you've already said you won't accept on air. The planned blind scoring wasn't needed, because Finding 2 settles the race-night question regardless of quality.

> **Upstream fault found in passing.** Baby's state notes sometimes contradict each other. In one beat Dustin is *"chasing Mike for first"* and, in the next note, *"behind Cade"*. Neither model can write a true line from contradictory facts. The Toddler brief must be built from one consistent race snapshot, not from notes that may have gone stale.

5. What this decides for Toddler

1. **Cloud Haiku is the race-night writer** for both call and analysis lines.
2. **Templates are the fast lane and the safety net.** They air instantly for fast events (the 21 OCT hybrid rule), and whenever the cloud is late, failing or unreachable.
3. **Toddler is still built for local models, but local is off by default on this PC.** The writer has three settings: cloud, local and hybrid. At startup it checks the free GPU memory and refuses local if there isn't about 3 GB free, rather than spilling into system memory mid-race.
4. **Short briefs are the standard request for every model.** Python works out the facts first: who, where, gap, what it means. The model only phrases them. That is what makes local usable on any machine with room, and it also keeps cloud prompts small and cheap.
5. **Lines are cleaned and checked in code before they air.** Strip speaker labels, keep one sentence, enforce the word cap, and check names and positions against the snapshot. Anything that fails is replaced by a template.
6. **No passages.** One speaker per beat, as White Paper 16 already sets out. Short single lines are faster on every model and safer to cut.

6. Getting local back on race night

Both real routes are hardware, not code:

Route	What it takes	What to expect
A second PC on the network	Any machine with an 8 GB-class GPU and no game running, serving Ollama to the broadcast PC.	Game-off speeds: about 0.35 s per line, plus a few milliseconds on the network. Toddler's writer only needs a different address.
A bigger GPU in this PC	12 GB or more, so F1 25, OBS and the model all fit. The i5-9400F CPU stays the bottleneck for everything else.	Should match game-off speeds; would need the same bench run to confirm.

Not recommended: a smaller local model (a 1B version) to squeeze onto the card. It would fit, but llama3.2 3B already gets facts wrong, and a smaller model will do worse.

7. Limits of this test

- OBS recorded for only the last few calls of the Toddler-format game run, and OBS's dropped-frame counts weren't captured. The OBS stutter is an eyewitness report, not a measurement.
- The Baby-format game-on run was stopped after 10 beats. Its numbers are directional, but a 20× slowdown doesn't need more samples.
- One local model was tested (llama3.2 3B, Ollama default quantisation) on one PC, with one race's data.
- Rule-break checks are mechanical. The accuracy findings in section 4 come from reading the lines against the facts, not from a scored sample.

8. Re-running the bench

From the repository folder in PowerShell (Ollama running; the cloud lane reads `ANTHROPIC_API_KEY`), or double-click `tools\Run LLM Bench.bat`:

`python tools\T13_LLM_Bench_V1_26OCT26.py --dry-run` builds the prompts and sends nothing.
`python tools\T13_LLM_Bench_V1_26OCT26.py --formats toddler --n 20` is the race-night check (game on track, OBS recording).
`python tools\T13_LLM_Bench_V1_26OCT26.py --n 40` is the full bench, both formats.

Results go to `hoover_bench_out\BENCH_<time>\`: `summary.txt`, `results.jsonl`, `gpu.csv`, and a blind scoring sheet with its key. Run it again whenever the hardware, the local model or the Toddler brief changes.